1) Performance

1.1) General overview:

When commenting on my model's performance I will mainly (unless stated otherwise) be referring to the performance it demonstrated when predicting crop yields for year 2022- the year where I have the largest amount of past data to train from, whilst also having actual crop yield results to compare. As to be expected, when selecting earlier years there is a proportionate **decrease** in **running time** as well as **predictive capability** due to there being fewer data instances used in the MLP training- making predictions for 2012, for example, means only training on 302 rows and as such the loss is substantially higher (on average by an order of magnitude for an individual crop vs 2022's figures) and variance is significantly less well accounted for (the average R^2 of a crop prediction for 2012 from my model was ~0.37 vs 2022's average of ~0.71).

The model demonstrated strong performance overall. It achieved a **Spearman's rank correlation of 0.991**, indicating near-perfect agreement between the predicted and actual yield rankings across countries. The **total absolute error (**$TAE = \sum_{i=1}^{n} |y_i - \widehat{y}_i|$**) amounted to approximately 34.83%** of the total actual yield in 2022, while the *average mean absolute percentage error (**mean APE) across crops was 48.91%**, with an average **median APE of 32.65%**** - this relatively large gap between mean APE and median APE and high mean APE indicates that the least accurate predictions skew the mean APE (see **1.2.2** for deeper inspection). The model explained a substantial portion of yield variance for most crops- if we consider an R^2 value above 0.55 as good and above 0.7 as excellent then **the model was excellent for approximately half of the crop types and at least good for over 80% of them.** However, several crops with limited data- such as Melonseeds and Tallowtree seeds -exhibited high error rates and lower R^2.

*Average mean absolute percentage error was calculated by taking the MAPE for each crop and then taking the mean average of these values. Average median APE was calculated by taking the median APE of each crop and taking the mean average of these values.

The stratification of each of my 2022 crop predictions with regard to how well they account for variance results is as follows:

$R^2\ Strata$	Count
$R^2 \geq 0.85$	35
$0.85 > R^2 \geq 0.7$	26
$0.7 > R^2 \geq 0.55$	24
$0.55 > R^2 \geq 0.4$	8
$0.4 > R^2$	9

Table 1: *Data taken from* `model_metrics_2022.csv` *- A csv file generated alongside the* `forecast_cv_results_2022.csv` *file, when running my final model* `forecast_with_cv_per_crop`. `Model_metrics_2022.csv` *provides analysis metrics for the predictions made in* `forecast_cv_results_2022.csv`.

1.2) Specific correlation and distance metrics:

1.2.1) ***Correlation Metric:***

I calculated **Spearman's rho** to be **0.991** when ranking the predicted yields of crops against the actual yields for 2022. This indicates that as actual crop yields increase, the predicted yields follow **nearly the exact same rank order**.
The correlation has **a p-value of 0.0**, suggesting that the relationship is statistically significant and extremely unlikely to have occurred by chance. **Figure 1** shows this information graphically:

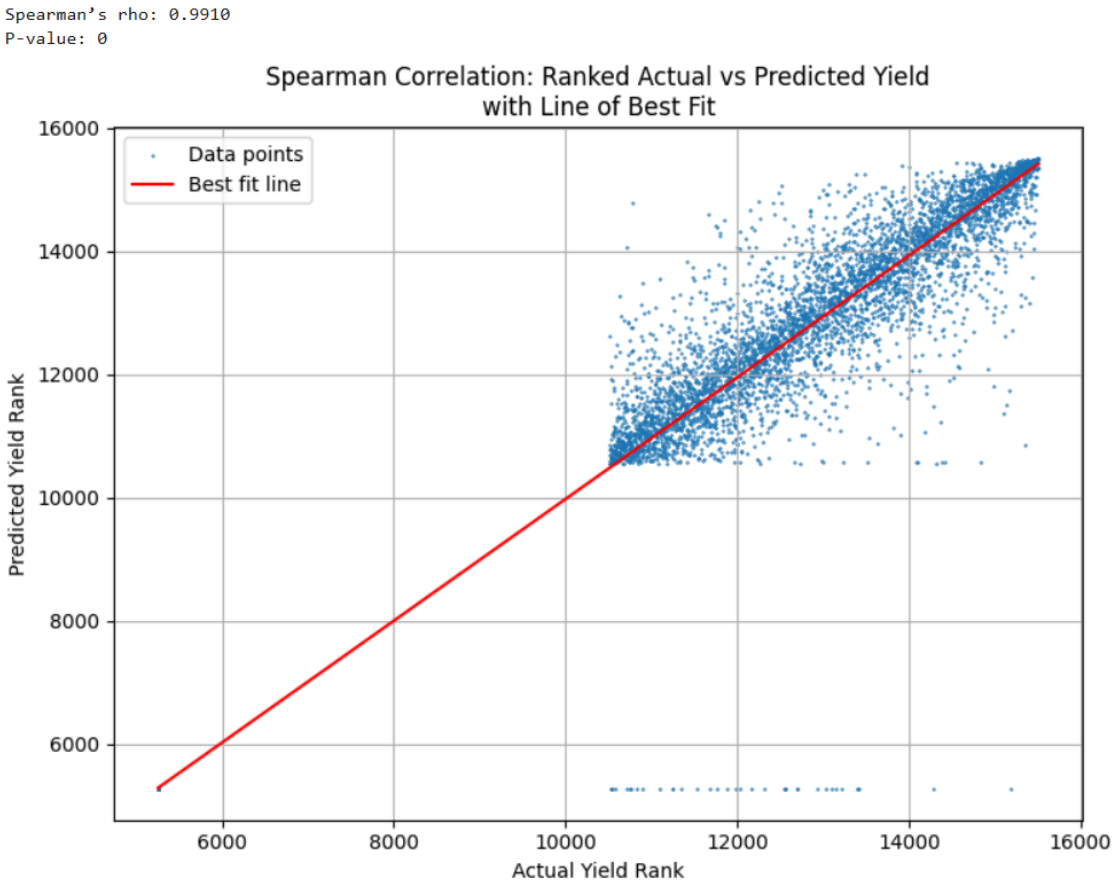

Figure 1 *- showing all data used to calculate Spearman Correlation.*

1.2.2) *Distance Metric:*

Root mean squared error (RMSE) was calculated for each crop yield prediction and the **mean average RMSE** was **2638.57** and **median average RMSE** was **1103.48** (indicating an upward skewing by larger errors). Given that the **average of all non-zero actual crop yields is $\overline{C}$ = 9828.88**, the scale of the average RMSE can be thought of in terms of the percentage of $\overline{C}$ to give an idea of the average error that is **less sensitive** to the noise produced from the few most inaccurate predictions (which skew the MAPE upwards). **The mean average RMSE is 26.8% of $\overline{C}$ and the median average RMSE is 11.2% of $\overline{C}$.**

While the RMSE averaged across crops is moderate at ~27%, the average MAPE is considerably higher at ~49%. This discrepancy suggests that while the model's overall error magnitude relative to typical crop yield is modest, **it exhibits significant variation at the individual prediction level** - particularly for crops with lower yields where percentage errors can inflate.

1.3) Data instances:

My complete usable data amounts to **2093 features** (all with 188 columns which represent latitude, longitude, year and then all monthly feature data provided as well as the land cover percentages) to predict **2585 Labels** (all with 102 values- one for each crop). However, there are 119 rows in my final dataframe (the product of merging my features and

labels by year and country) with features and no corresponding yield data, as well as 611 rows with yield data and no corresponding features. So my functional data that is used in the MLP training after dropping those rows is: **1974 instances with 290 attribute columns**. When making a prediction for 2022, only feature data from prior to 2022 is used, amounting to **1822 instances**.

If a different target year is given, below 2022, the amount of features and labels decrease accordingly so that my model can simulate being in the target year- the only feature data trained from is up to and **not** including the target year. This training is then used to generate a prediction for crop yields for all countries for that year (with feature and yield data) which is then compared to actual crop yield values from that year. Similarly, if more data was available- such as environmental data and yield data beyond 2022- my model's data instances would increase accordingly to incorporate them. If 2023 is the target year a prediction is provided based on 1974 data instances, but there is no validation possible.

1.4) Training:

I used 5-fold **cross-validation (CV)** to evaluate **8 different hyperparameter combinations**, resulting in a total of 40 multilayer perceptron (MLP) models being trained and validated. In each fold, 4 out of 5 partitions of the training data were used to train a model, while the remaining partition was used to validate it. This process was repeated so that each fold served as the validation set once. For each hyperparameter combination, the average negative root mean squared error (RMSE) across the 5 validation folds was computed. The combination with the highest average score (i.e., the least negative RMSE, which corresponds to the lowest actual RMSE) was then selected. A final MLP was trained using this best hyperparameter configuration on the full training data available for that crop and year.

I would like to have used 10 folds as I believe that is industry standard and would only serve to increase my model's predictive capabilities, but -having trained using primarily 3 folds for the majority of my testing and 5 folds to produce my final predictions- I saw only a marginal increase in performance going from 3 to 5 folds, yet a significant increase in time to run my final `forecast_with_cv_per_crop` function, that made any further increase in CV folds unfeasible for my laptop.

2) Model

2.1) Specifications and choices:

The choice of CV regression with 5 folds and 8 combinations of hyperparameters was to balance performance with time consumption.

The choice of using two hidden layers was based on prototype cross-validation models, where two-layer architectures consistently outperformed single-layer ones. This is expected if crop yield is influenced by complex interactions between features - such as specific combinations of rainfall, temperature, and latitude (the latter acting as a proxy for other environmental factors like sunlight hours and average temperature).

Hidden layer sizes were chosen as powers of 2 to align with common hardware optimisations in binary-based computing for faster and more efficient calculations.

From numerous tests there was consistently a fairly even split as to which architecture- (128,64) or (256,128)- was optimal for a given crop, so the choice to include both in my grid of combinations was made

Initially I chose 0.001 as a starting L2 regularisation strength (α) out of convention to reduce overfitting without inhibiting learning and then used a preview version of my final function in my pipeline (where I can set how many crops are predicted) with all other hyperparameters set to a single value and 10 CV folds, but with 5 choices of α: [0.00005, 0.0001, 0.0005, 0.001, 0.002]. Then by taking into account which values for α performed the best (highest negative RMSE) I chose 0.0005 and 0.0001 to be part of my final function.

For the initial learning rate I adopted the same method- set all other hyperparameters and provided a selection of possible learning rates- [0.0001, 0.0003, 0.0005, 0.001, 0.003]- for a 10 fold CV regression model. I then selected 0.0001 and 0.001 to be part of my final function as the two highest performers.

I set the maximum number of iterations to 5000, with early stopping enabled: if the loss failed to improve by more than 0.0001 over 30 consecutive iterations, training would halt and proceed to the next crop. A limit of 5000 ensured convergence for nearly all crops, with only rare exceptions. Early stopping was introduced to reduce training time, and the threshold of 30 iterations was chosen after testing 10 and 20 - which often led to premature termination, likely due to the model getting stuck in local minima. Increasing this threshold to 30 mitigated the issue.

2.2) Avoiding overfitting:

To help avoid overfitting, I used 3 and 10-fold CV during model selection and prototyping. And to produce my final yield predictions I used a 5-fold CV function. Averaging performance across folds ensures that the model generalises well across different subsets of data, rather than fitting noise in any one split.

In addition to CV, I employed two techniques directly aimed at reducing overfitting during training: L2 regularisation and early stopping. L2 regularisation was used to reduce overfitting by penalising large weight values, which helps the model learn smoother mappings and reduces sensitivity to noise in the training data.

Early stopping was implemented to halt training if the validation loss did not improve by more than 0.0001 for 30 consecutive iterations. This prevents the model from continuing to train past the point of generalisation, thereby avoiding performance degradation due to overfitting.

Due to GridSearchCV not supporting tracking validation loss history I cannot plot a validation loss vs training loss curve with my actual final data but instead using one of my preview functions I can run selections of crops with set hyperparameters and track their learning curves- using Maize as an example with parameters set to match our forecast_with_cv_per_crop function for Maize I was able to produce this learning curve:

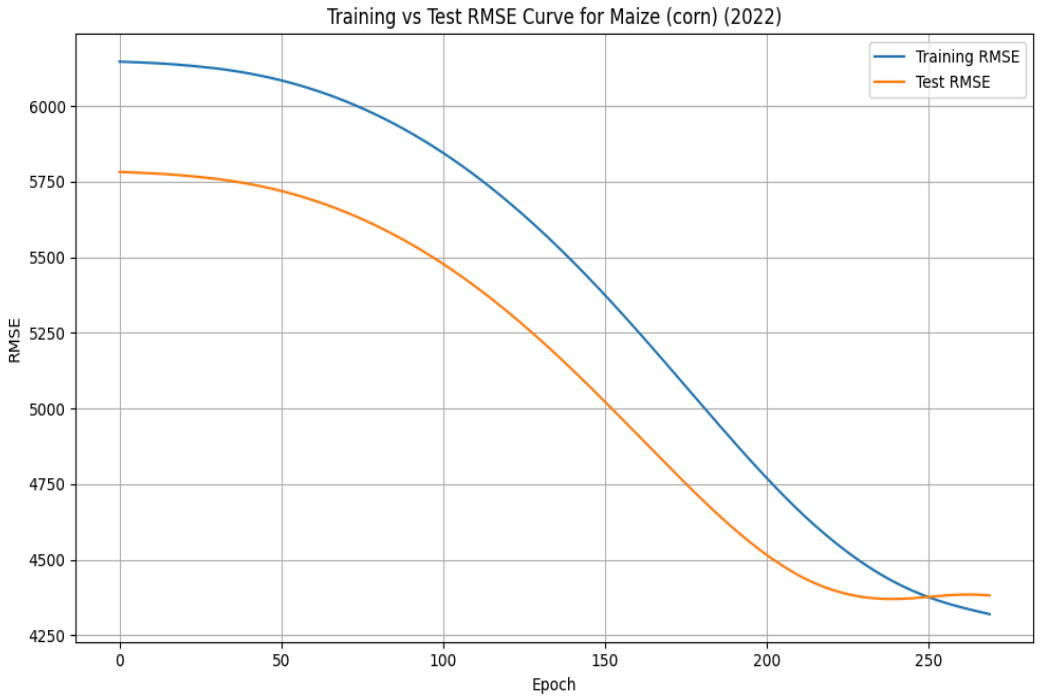

***Figure 2** - showing an example learning curve for Maize (Corn)*

This pattern of training and test RMSE tracking closely and reaching a similar low point is consistent through most of my training vs test data. **The mean absolute difference of the training RMSE and the test RMSE for all crops after the final training iteration** was just **496.40**, with the **median** being **135.42**. Given that the average non-zero crop yield- $\overline{C}$**-** was 9828.22, **these differences represent just ~5% and 1.3% error** for 2022 respectively- This consistent minimal gap between training and test RMSE indicates that the

model generalises well to unseen data, and is not over or under-fitting. A brief summary of such metrics can be seen in the table below:

	Train RMSE	Train R^2	Test RMSE	Test R^2	Final Loss	Train Test RMSE difference
Mean	2425.970	0.741	2638.574	0.714	9351518.247	496.408
Median	968.1578228	0.791	1103.479	0.751	422572.445	135.425
St Dev	3730.912966	0.219	3910.597	0.219	29838351.74	833.472

Table 2- showing some key metrics taken from analysis of *`model_metrics_2022.csv`*

3) Features & Labels

3.1) Main overview:

I decided my labels to be each crop for each country each year. So 102 crops for 2585 Country Year combinations- as our stated goal was to predict crop yields for a given year and geographical location- I made the decision that countries would be my geographical locations, as that was the resolution provided by the yield data. (It would not have been feasible to predict per city).

Using centroid information from `country_latitude_longitude_area_lookup(in).csv` I labelled all of the available feature tables provided `(CanopInt_inst_data(in).csv, ESoil_tavg_data(in).csv, Rainf_tavg_data(in).csv, Snowf_tavg_data(in).csv, SoilMoi0_10cm_inst_data(in).csv, SoilMoi10_40cm_inst_data(in).csv, SoilMoi40_100cm_inst_data(in).csv, SoilMoi100_200cm_inst_data(in).csv, SoilTMP0_10cm_inst_data(in).csv, SoilTMP10_40cm_inst_data(in).csv, SoilTMP40_100cm_inst_data(in).csv, SoilTMP100_200cm_inst_data(in).csv, TVeg_tavg_data(in).csv, TWS_inst_data(in).csv, Land_cover_percent_data(in).csv)` (I will hereforth refer to these files as the feature files) by appending a country column. If a row in the feature data had a latitude and longitude that fell within a particular country's centroid $\pm$ the centroid radius of that country, then that row would have that country added to its new Country column. This of course led to multiple rows of features per country year, especially for larger countries. To match our yield resolution- 1 row = 1 country in a given year- I took the mean average over all the feature rows with the same country and year label and merged all of these features by their country and year columns- giving me a final table of numerical feature values of size 2093x188.

As mentioned in **1.3**, upon merging and dropping missing rows, `forecast_with_cv_per_crop` with a target year set to 2022 utilised 1822 instances of 188 features to predict a label vector of length 102 for each country.

Given that crop growth is a physical phenomenon strongly influenced by environmental conditions, it was clear that the data from the feature files would be relevant. However, the decision to retain the data in its per-month format, and to include variables such as year, average latitude, and average longitude in the feature set used to train the MLPs, required more careful consideration.

3.2) Rationale and niche cases:

I chose to preserve the monthly resolution because aggregating features (e.g. rainfall) into a yearly total or average could obscure important temporal dynamics. For instance, if one country experienced a monsoon for a single week followed by drought, while another had steady rainfall throughout the year - both might have the same total rainfall, but there are vastly different implications for crop yield for each. Such critical differences would be lost in aggregation but are likely essential to accurate prediction.

Latitude and longitude can serve as proxies for physical variables relevant to crop growth — such as sunlight hours, climate, and, less obviously, gravitational intensity. Many crops depend on upward growth during part of their life cycle, and the variation in apparent

gravity caused by Earth's rotation may have a subtle but non-negligible impact. Due to centrifugal force, apparent weight is reduced by approximately 0.5% at the equator (latitude 0°) compared to the poles. This gradient may influence certain biological or physiological processes, and thus contribute marginally to differences in yield across latitudes.

Similarly, Year could capture other less obvious temporal information- eg, if there is a cultural shift one year away from certain products that require a given crop and thus a reduced demand for it- and less of it planted. Having run my `forecast_with_cv_per_crop` function with Year, (average) Latitude and (average) Longitude included and excluded, my theory seems to bore out as the results with them all included were on average marginally better (less total loss).

4) Preprocessing

4.1) Analysis:

Upon merging all of my averaged feature files into one file with only 1 row per country per year it became apparent that there was a sparse amount of missing entries, for example Afghanistan missing August soil moisture at 100cm-200cm depth. I conducted a scan for non numerical entries within my feature table and found **483** (in my grid of 2093x189 this represented ~0.12% of my feature data points). Given the small number and proportion of missing values I opted for the relatively more computationally taxing option to do a **KNN imputation** so as not to artificially reduce variance (as would be the case with just using the mean).

4.2) KNN to impute:

I chose my number of nearest neighbours (value of K) to be [1, 3, 5, 7, 10] and a masked fraction of 5%. By evaluating the RMSE produced from each K value I was pointed towards 3, 5 and 7 as optimal but as the results were close and sparse I opted to refine. I made a function that would test 50 random seeds checking K values of [3,4,5,6] and counting the amount of times each value led to the smallest RMSE. I reduced my mask to 0.12% (to better reflect the imputation task at hand) and over many runs 4 was consistently the best performer. I then used KNN imputation with **4 as my value for K** to Impute my feature table so that there were no more missing values. This completed my feature data preprocessing.

4.3) Data extraction:

For my label data, I extracted the year, country, yield value, and crop type from `Yield_and_Production_data(in).csv` to construct `sorted_yield_data.csv`. This file contains one row per country-year pair, with columns for Country, Year, and 102 crops listed in alphabetical order. Each crop column contains the total yield for that crop, corresponding to the country and year specified in that row.